

Raport profesionist de verificare și remediere

Analiză a modificărilor realizate de Vibe Coding pentru proiectul Vibe Caffè
Vibe Caffè Website | 16 aprilie 2026

1. Contextul solicitării

Am primit sarcina de a verifica modificările realizate de Vibe Coding după un set de recomandări furnizate anterior în format HTML. Scopul a fost dublu: audit tehnic al implementării și, acolo unde era necesar, remediere completă a problemelor descoperite.

Analiza a pornit de la trei intervenții raportate de Vibe Coding: transformarea numărului din footer în link telefonic, adăugarea unui endpoint POST pentru rezervări și introducerea unui bloc de contact flotant cu telefon și WhatsApp.

2. Ce a implementat corect Vibe Coding

- Numărul din footer a fost transformat într-un link telefonic funcțional.
- A fost creată componenta FABContact pentru butoane flotante de contact.
- FABContact a fost integrată în layout-ul global al aplicației.
- A fost adăugat un endpoint POST în ruta /api/rezervari.

3. Problemele identificate în urma auditului

- Endpointul nou POST /api/rezervari nu era folosit de formularul public; pagina publică scria în continuare direct în Supabase.
- Implementarea POST avea o incompatibilitate între statusurile folosite în cod și schema reală a bazei de date.
- Exista nealiniere între migrarea locală, codul aplicației și schema live din Supabase pentru coloana status din tabelul rezervari.
- Panoul admin trata inconsistent statusurile istorice salvate în română și engleză.
- Migrarea Supabase din 2026-04-09 nu era idempotentă și bloca aplicarea altor migrații în live.
- Fișierele atinse de intervenție nu erau complet curate la nivel de lint înainte de predare.

4. Remedierile efectuate

- Am mutat fluxul public de rezervări pe endpointul API, astfel încât pagina /rezervari să folosească POST /api/rezervari în loc de insert direct în Supabase.
- Am corectat endpointul /api/rezervari pentru a include tenant_id și pentru a rămâne aliniat cu structura actuală a aplicației.
- Am unificat interpretarea statusurilor în panoul admin, astfel încât valorile vechi și noi să fie afișate coerent în filtre, badge-uri și acțiuni.
- Am curățat app/admin/page.tsx până la nivel lint-clean, eliminând erori și warning-uri relevante.
- Am creat migrarea 20260416_fix_rezervari_status_constraint.sql pentru a repara schema live a coloanei status.
- Am făcut migrarea 20260409_add_rls_policies.sql sigură de rerulat prin introducerea de DROP POLICY IF EXISTS înainte de recrearea politicilor.
- Am aplicat migrațiile în Supabase live prin supabase db push.

5. Verificări și validări realizate

- Am verificat fișierele modificate de Vibe Coding prin diff și analiză directă în cod.
- Am rulat lint pe fișierele corectate până la obținerea unui rezultat curat pentru zonele afectate.
- Am rulat build complet Next.js; buildul final a trecut fără erori.
- Am testat endpointul /api/rezervari local pe buildul aplicației.
- Am diagnosticat direct baza Supabase live prin interogări REST pentru a confirma valorile reale din tabelul rezervari.
- Am confirmat end-to-end că o rezervare nouă se salvează corect în baza live după aplicarea migrării.

6. Rezultatul final confirmat

După remediere și migrare, fluxul public de rezervări funcționează cap-coadă. Endpointul POST /api/rezervari răspunde cu succes: true, iar rezervarea creată apare în Supabase cu tenant_id corect și statusul „în așteptare”.

Panoul admin tratează acum coerent statusurile, iar fișierele principale implicate în intervenție sunt curate și verificabile.

- Build final: trecut cu succes.
- Lint pentru fișierele cheie: trecut.
- Migrații Supabase: aplicate în live.
- Test final API: trecut.
- Persistență în baza live: confirmată.

7. Fișiere și zone modificate de mine

- app/api/rezervari/route.ts
- app/rezervari/page.tsx
- app/admin/page.tsx
- supabase/migrations/20260409_add_rls_policies.sql
- supabase/migrations/20260416_fix_rezervari_status_constraint.sql

8. Concluzii pentru Vibe Coding

Implementarea inițială a avut elemente bune la nivel de interfață și direcție, dar a fost incompletă la nivel de integrare și verificare operațională. Principala lecție este că o modificare raportată ca „implementată” trebuie validată nu doar vizual, ci și pe fluxul real, în baza de date reală și în administrația reală a aplicației.

Pentru intervenții viitoare, recomandarea este ca orice schimbare care introduce un endpoint nou sau atinge o schemă de date să fie însoțită obligatoriu de: integrare în UI, verificare build, verificare lint, test API real și verificare pe datele live sau într-un mediu cât mai apropiat de live.

9. Mesaj scurt pentru prezentare

Vibe Coding a implementat corect partea vizibilă a cerinței, dar au rămas probleme de integrare și consistență între frontend, API și baza de date. Aceste probleme au fost identificate, corectate și validate end-to-end, inclusiv prin aplicarea unei migrări în Supabase live.